

JavaAgent CLI 伪代码文档

本文档用自然语言描述整体架构和核心流程，不包含任何可运行代码。

包结构总览

```

com.javagent
├─ JavaAgentCLI          #
├─ BannerPrinter         #
├─ SlashCommandCompleter # Tab
├─ core/                 #
│   ├─ Agent             # Agent
│   ├─ Config             #
│   ├─ ConversationManager #
│   ├─ ApprovalManager    #
│   ├─ ApprovalHandler    #
│   ├─ ApprovalDecision   #
│   ├─ ApprovalOutcome    #
│   ├─ ContextUsage       # token
│   └─ ToolStats          #
├─ model/                #
│   ├─ Role               #
│   ├─ Message            #
│   ├─ ToolCall           #
│   ├─ ToolResultMessage  #
│   ├─ ModelResponse      #
│   ├─ ModelClient        #
│   ├─ MockModelClient    #
│   ├─ OpenAiCompatibleModelClient # OpenAI
│   ├─ TextStreamHandler  #
│   └─ ToolDisplayCallback #
├─ tools/                #
│   ├─ Tool               #
│   ├─ ToolDefinition     #
│   ├─ ToolExecutionResult #
│   ├─ ToolRegistry       #
│   ├─ ToolProvider       # SPI
│   ├─ FileToolSupport    #
│   ├─ ReadFileTool       #
│   ├─ GrepTool           #
│   ├─ ListDirectoryTool  #
│   ├─ WriteFileTool      #
│   ├─ EditTool           #
│   ├─ DeleteFileTool     #
│   ├─ BashTool           # Shell
│   └─ NetworkTool        # HTTP
└─ util/                 #
    ├─ Terminal           # ANSI + splitLines
    ├─ TokenCounter        # jtokkit token
    ├─ ContextDisplay      # /context
    ├─ MarkdownRenderer    # Markdown
    ├─ Sanitizer           #
    └─ RateLimiter         #

```

一、com.javagent 包

JavaAgentCLI —— 程序入口

****作用****: 整个应用的启动类，负责组装各组件并启动命令行交互循环。

****核心字段****:

- 配置对象、对话管理器、工具注册中心、模型客户端、Agent 引擎
- JLine3 LineReader (终端行编辑、历史翻页)
- AtomicBoolean awaitingApproval (审批时暂停 spinner)

****方法流程****:

main

- 创建 JavaAgentCLI 实例，调用 run 方法

run

1. 加载默认配置 (从 config.properties 文件)
2. 解析命令行参数 (--mock、--real、--api-key、--help)
3. 初始化/重建运行时组件
4. 启动命令行交互循环

applyArgs

- 遍历命令行参数数组
- 遇到 --mock 则切换为模拟模式
- 遇到 --real 则切换为真实模式
- 遇到 --api-key 则设置 API 密钥
- 遇到 --help 则打印帮助并退出

rebuildRuntime

1. 如果对话管理器为空则创建新实例 (已存在则复用)
2. 创建工具注册中心，依次注册：读文件、搜索、目录列表、写文件、编辑、删除文件
3. 如果 Bash 已启用，注册 Bash 工具
4. 注册 Network 工具
5. 通过 SPI (ServiceLoader) 加载外部 ToolProvider 插件
6. 根据当前模式选择模型客户端 (模拟模式用 Mock，真实模式用 OpenAI 兼容客户端)
7. 创建 Agent 实例，传入配置、模型客户端、工具注册中心、对话管理器

startCli

1. 打印方框风格启动横幅 (BannerPrinter)
2. 显示当前模式、模型、工具数量
3. 尝试恢复上次的会话
4. 进入主循环：
 - 显示彩色提示符 `>`
 - 读取用户输入 (JLine3 LineReader)

- 如果输入为空则跳过
- 如果以 `/` 开头则处理命令
- 否则交给 Agent 处理，传入审批回调和流式输出处理器
- 根据流式输出是否已触发，决定直接打印还是换行

handleCommand

- 解析命令和参数
- `/help` → 打印帮助（18 个命令）
- `/exit` 或 `/quit` → 退出程序
- `/clear` 或 `/new` → 开始新会话
- `/save` → 保存当前会话
- `/load` → 加载指定会话（按 ID 或标题）
- `/sessions` → 列出所有保存的会话
- `/tools` → 列出可用工具（8 个）
- `/mode` → 切换模式（mock/real），切换后重建运行时
- `/stream` → 开关流式输出
- `/bash` → 开关 Bash 工具，切换后重建运行时
- `/bypass` → 跳过所有工具审批确认
- `/prompt` → 查看/设置/重置自定义系统提示词
- `/approvals` → 查看审批缓存条目数或清空缓存
- `/reload` → 运行时重载配置文件
- `/export` → 导出当前对话为 Markdown
- `/stats` → 查看工具执行统计
- `/network` → 网络请求工具
- `/context` → 查看上下文 token 占用（彩色柱状图）
- `/compact` → 压缩对话历史（LLM 摘要，节省 token）
- `/effort` → 调节模型思考强度
- `/status` → 显示当前状态信息

promptApproval

1. 暂停 Braille spinner (awaitingApproval.set(true))
2. 打印工具调用信息（工具名、参数）
3. 提示用户输入 yes/no/cancel
4. 根据用户输入返回对应的审批决策
5. 恢复 Braille spinner (awaitingApproval.set(false))

ConsoleTextStreamHandler (内部类)

- 实现 TextStreamHandler 接口
- 每收到一块文本就立即打印到控制台
- 记录是否已输出过文本，用于判断是否需要额外换行

二、com.javagent.core 包

Agent —— 核心引擎

****作用****: 整个系统的"大脑", 协调各组件完成用户请求, 实现 Agent Loop (智能体循环)。

****核心字段****:

- 配置、模型客户端、工具注册中心、对话管理器、审批管理器、工具统计

processTurn (无流式)

- 调用带流式参数的版本, 传入 null

processTurn (核心方法)

1. 把用户消息加入对话历史
2. 构建系统提示词 (告诉 AI 它是谁、有哪些工具可用、编辑工具使用规范)
4. 进入 Agent Loop (最多循环 maxIterations 次) :
 - 根据配置决定是否启用流式输出
 - 调用模型客户端发送请求, 获取响应
 - 如果是错误响应:
 - 如果错误包含 "reasoning_content", 自动清理旧上下文并重试
 - 记录错误消息, 自动保存, 返回错误文本
 - 如果是纯文本响应 → 记录 AI 回复 (含 reasoningContent), 自动保存, 返回回复文本
 - 如果是工具调用响应 → 记录 AI 的工具调用消息, 逐个执行工具调用:
 - 调用 displayCallback.onToolStart 通知 UI
 - 执行工具, 记录 ToolStats 统计
 - 同一工具连续失败 3 次 → 中断循环
 - 工具结果经 Sanitizer 脱敏后加入对话历史
 - 调用 displayCallback.onToolEnd 通知 UI
 - 继续循环
5. 超过最大循环次数 → 记录提示消息, 自动保存, 返回提示

executeToolCall

1. 从工具注册中心查找工具, 找不到则返回错误
2. 通过审批管理器检查是否被授权
3. 审批不通过则返回拒绝原因
4. 审批通过则执行工具, 记录统计, 返回执行结果

buildSystemPrompt

1. 设定 AI 身份 ("JavaAgent CLI, 简洁的编程助手")

2. 添加使用原则（按需使用工具、工作区内操作、避免破坏性操作）
3. 添加编辑工具使用规范（old_string 必须精确匹配）
4. 如果有自定义提示词则追加
5. 列出所有可用工具的名称、描述、别名和必填参数

compact

1. 检查消息数量是否足够（至少 4 条）
2. 保留最近 6 条消息，其余发送给 LLM 做摘要
3. LLM 生成中文摘要（保留关键决策、重要事实、用户核心需求）
4. 构建新消息列表：摘要 + 最近消息
5. 替换对话历史，返回压缩结果（token 节省量）

contextUsage

1. 计算系统提示词 token 数（缓存）
2. 计算工具定义 token 数（缓存）
3. 计算消息 token 数（实时）
4. 返回 ContextUsage 数据模型

autoSaveQuietly

- 如果配置了自动保存，则保存当前会话
- 保存失败时静默忽略（写日志），不影响主流程

Config —— 配置管理器

****作用****：管理应用的所有配置项，支持从 properties 文件加载和保存。

****配置项****：

配置项	说明	默认值
----- ----- -----		
agent.mock_mode	是否使用模拟模式	true
agent.api_key	API 密钥	空
agent.base_url	API 基础 URL	https://api.openai.com/v1
agent.model	模型名称	gpt-5.4-mini
agent.auto_save	是否自动保存会话	true
agent.max_iterations	最大工具调用循环次数	12
agent.enable_bash	是否启用 Bash 工具	false
agent.stream_responses	是否流式输出	true
agent.system_prompt	自定义系统提示词	空
agent.approval_cache	是否启用审批缓存	true
agent.allow_external_paths	是否允许访问工作区外路径	false

agent.bypass_permissions	是否跳过所有工具审批确认	false
agent.effort	推理深度	medium
agent.max_tokens	上下文窗口 token 数	200000
agent.compact_threshold	自动压缩阈值	0.8
agent.rate_limit_qps	API 请求速率限制（每秒请求数）	10

****方法流程**:**

loadDefault

- 工作目录设为当前目录，应用主目录设为 ~/.javaagent-cli
- 调用内部加载方法

load

- 从文件读取配置 → 填充缺失的默认值 → 保存回文件

****配置文件查找顺序**:**

1. 项目根目录（有 pom.xml 的目录）
2. 工作目录
3. ~/.javaagent-cli/config.properties

****路径方法**:**

- stateDirectory: 状态目录（优先在工作目录下创建 .javaagent-cli，否则放 appHome）
- sessionPath: 旧版会话文件路径
- sessionDirectory: 会话存储目录
- currentSessionMarkerPath: 当前会话标记文件

ConversationManager —— 对话管理器

****作用**:** 管理对话历史和多会话，支持保存/加载/切换不同对话。

****核心字段**:**

- 配置对象、Jackson 序列化器
- 当前会话 ID、标题、消息列表、开始时间、最后一条 AI 回复

****方法流程**:**

构造器

1. 保存配置引用
2. 初始化 Jackson 序列化器（支持 Java 时间类型、日期用字符串、格式化输出）
3. 开始一个新的空会话

addUserMessage / addAssistantMessage / addAssistantToolCallMessage / addToolResultMessage

- 分别创建对应角色的消息对象，加入当前消息列表

replaceMessages

- 替换当前消息列表（用于 compact 压缩后更新）

currentContext

- 返回当前消息列表的不可变副本

startNewSession

1. 生成新的会话 ID (UUID)
2. 设置标题（传入标题或默认标题）
3. 清空消息列表
4. 重置会话开始时间和最后回复

saveCurrentSession

1. 创建会话快照（包含 ID、标题、开始时间、更新时间、消息列表、最后回复）
2. 确保目录存在
3. 保存到三处：会话目录下的 JSON 文件、旧版路径、当前会话标记文件

loadLastSession

1. 优先读取标记文件，获取会话 ID
2. 如果找到对应的会话文件则加载
3. 否则回退到旧版路径
4. 加载后应用快照恢复会话状态

loadSession

1. 列出所有保存的会话
2. 按 ID 或标题模糊匹配
3. 如果是 "latest" 则加载最新的
4. 找到则加载快照并更新标记文件

listSessions

- 遍历会话目录下的 JSON 文件
- 提取每个会话的摘要信息
- 按更新时间倒序排列

exportToMarkdown

- 将当前对话导出为 Markdown 格式

****内嵌数据结构**：**

- SessionSummary：会话摘要（ID、标题、更新时间、消息数）
- SessionSnapshot：会话快照（完整会话数据，用于保存/加载）

ApprovalManager —— 审批管理器

****作用****：控制工具是否需要用户确认才能执行，实现安全策略和审批缓存。

****核心字段****：

- 配置对象、审批缓存（Map 结构）

authorize

1. 如果 bypass 模式开启 → 直接返回批准
2. 获取工具定义
3. 执行策略检查（evaluatePolicy）
4. 如果策略结果为"允许" → 直接返回批准
5. 如果策略结果为"拒绝" → 直接返回拒绝
6. 如果策略结果为"需审批" → 检查缓存
7. 缓存命中 → 沿用缓存的审批决策
8. 缓存未命中 → 调用审批回调请求用户确认
9. 用户决策不为"取消"时存入缓存
10. 返回审批结果

evaluatePolicy（策略检查）

1. 如果是 Bash 工具且未启用 → 拒绝
2. 提取工具参数中的路径，如果路径在工作区外且不允许外部路径 → 拒绝
3. 如果是破坏性工具且路径为受保护路径（.git、javaagent-cli 等） → 拒绝
4. 如果是删除工具且目标是工作区根目录或目录 → 拒绝
5. 如果工具标记为不需要审批 → 允许
6. 其他情况 → 需要用户审批

extractPath

- 从工具参数中提取路径（大多数工具用 "path" 参数，Bash 用 "workingDirectory"）
- 将路径解析为绝对路径

isProtectedPath

- 检查路径是否位于状态目录、配置文件、.git 目录、javaagent-cli 目录等受保护位置

cacheKey

- 生成缓存键：工具名 + 标准化后的参数（路径参数会解析为绝对路径后排序）

ApprovalDecision —— 审批决策枚举

****作用****：定义用户对工具调用的三种审批结果。

- APPROVED: 同意执行
- DENIED: 拒绝执行
- CANCELLED: 取消 (中止操作)

ApprovalOutcome —— 审批结果

****作用****: 封装一次审批的最终决定, 包含是否批准、是否来自缓存、审批理由。

****工厂方法****:

- approved / cachedApproved: 创建批准结果 (区分是否来自缓存)
- denied / cachedDenied: 创建拒绝结果 (区分是否来自缓存)

ApprovalHandler —— 审批回调接口

****作用****: 向用户请求审批的抽象接口, 实现依赖倒置。

- 唯一方法 request(ToolCall): 接收工具调用请求, 返回审批决策
- 标记为函数式接口, 可用 Lambda 创建

ToolStats —— 工具执行统计

****作用****: 追踪每个工具的调用次数、总耗时、错误次数、平均耗时。

****核心数据结构****: ConcurrentHashMap<String, StatsEntry>

record

- 记录一次工具执行 (工具名、耗时、是否出错)

format

- 格式化输出统计表格

StatsEntry

- calls: 调用次数
- totalMs: 总耗时
- errors: 错误次数
- avgMs: 平均耗时

三、com.javagent.model 包

Role —— 角色枚举

****作用****: 对话消息的身份标识。

- SYSTEM: 系统提示词, 告诉 AI 如何行为
- USER: 用户输入的消息
- ASSISTANT: AI 的回复
- TOOL: 工具执行后返回的结果

每个枚举值持有 `apiValue` 字段, 对应 API 协议中的小写字符串 (如 "user")。

Message —— 消息记录

****作用****: 系统中最核心的数据结构, 每一轮对话产生一条消息。

****字段****: ID、角色、内容、时间戳、工具调用列表、工具执行结果、`reasoningContent`

****约束****: 一条消息不会同时有 `toolCalls` 和 `toolResult`。

****工厂方法****:

- `system / user / assistant`: 创建纯文本消息
- `assistantWithToolCalls`: 创建包含工具调用的 AI 消息
- `toolResult`: 创建工具结果消息 (区分成功和失败)

****辅助方法****:

- `hasToolCalls`: 是否包含工具调用
- `isToolError`: 工具执行是否出错

ToolCall —— 工具调用请求

****作用****: AI 决定调用某个工具时创建的对象。

****字段****: 调用 ID、工具名称、工具参数

****工厂方法****: `of(name, input)` —— 自动生成 ID 创建实例

ToolResultMessage —— 工具执行结果消息

****作用****: 工具执行完成后封装到 `Message` 中的结果。

****字段****: 对应的工具调用 ID、工具名称、是否出错、结果内容

****工厂方法****: `success / error` —— 分别创建成功和失败的结果

ModelResponse —— 模型响应

****作用****: AI 模型返回的完整响应, 有三种类型。

****响应类型****:

- TEXT: 纯文本回复

- TOOL_CALLS: 请求调用工具
 - ERROR: 请求出错
- **字段****: content、toolCalls、errorMessage、reasoningContent
- **工厂方法****: text / toolCalls / error —— 创建对应类型的响应
- **辅助方法****: isText / isToolCalls / isError —— 判断响应类型

ModelClient —— 模型客户端接口

****作用****: 与 AI 模型通信的统一入口，面向接口编程以支持多种模型。

****方法****:

- chat (核心) : 发送对话请求，返回模型响应
- chat (带流式输出) : 默认实现，先获取完整响应，再将文本分块发送给流式处理器
- name: 返回客户端名称

****分块策略****: 文本长度的 1/6，最少 12 字符，最多 32 字符

MockModelClient —— 模拟模型客户端

****作用****: 不需要 API Key 的本地测试客户端，通过关键词匹配模拟 AI 行为。

chat

1. 如果消息列表为空 → 返回就绪提示
2. 如果最后一条是工具结果 → 总结工具执行结果
3. 找到最后一条用户消息
4. 关键词匹配:
 - "删除/delete" → 返回 delete_file 工具调用
 - "写入/write" → 返回 write_file 工具调用
 - "编辑/edit/修改/替换" → 返回 edit 工具调用
 - "列出/目录/ls" → 返回 list_directory 工具调用
 - "读取/read" → 返回 read_file 工具调用
 - "搜索/grep" → 返回 grep 工具调用
 - "bash/shell" → 返回 bash 工具调用
 - "网络/http/fetch" → 返回 network 工具调用
 - "help/工具" → 返回帮助文本
5. 未匹配 → 返回默认文本提示

summarizeToolResult

- 根据工具名称 (read_file、grep、list_directory、edit 等) 生成不同格式的总结
- 截取结果的前若干行

OpenAiCompatibleModelClient —— OpenAI 兼容客户端

****作用****: 调用真实的 AI API, 支持所有 OpenAI 格式兼容的模型服务, 支持 SSE 流式输出。

chat

1. 检查 API Key 是否已配置, 未配置则返回错误
2. 应用速率限制 (RateLimiter)
3. 构建 JSON 请求体 (含 stream: true)
4. 发送 HTTP POST 请求到 API 端点
5. 如果启用流式输出 → 逐行解析 SSE 事件流
6. 如果未启用流式输出 → 解析完整 JSON 响应
7. 检查 HTTP 状态码, 400+ 尝试降级重试 (去除 tools 参数)
8. 解析响应, 转换为 ModelResponse
9. 如果 API 因 reasoning_content 报错, 返回特定错误信息

buildRequestBody

1. 设置模型名、温度、工具选择策略
2. 构建消息数组: 系统提示词 + 对话历史
3. 根据消息角色分别处理: USER/SYSTEM 只设内容, ASSISTANT 可能含工具调用和 reasoningContent, TOOL 需配对工具调用 ID
4. 构建工具定义数组: 每个工具包含名称、描述、参数 Schema、必填参数列表

SSE 流式处理

1. 使用 HttpClient.BodyHandlers.ofLines() 逐行读取
2. 解析每个 "data:" 行的 JSON
3. 提取 delta.content 并实时调用 streamHandler.onChunk()
4. 增量解析工具调用的 id/name/arguments (ToolCallDelta)
5. 提取 reasoning_content (thinking 模型)
6. [DONE] 事件结束流

parseResponse

1. 检查响应中是否有错误对象
2. 提取第一个 choice 的 message
3. 如果 message 中有 tool_calls 字段 → 解析为工具调用响应
4. 如果有 reasoning_content → 提取推理内容
5. 否则 → 提取文本内容作为纯文本响应

TextStreamHandler —— 流式输出接口

****作用****: 用于逐块输出 AI 的回复, 提升用户体验。

- 函数式接口，唯一方法 onChunk(chunk)
- 可用 Lambda 创建

ToolDisplayCallback —— 工具执行显示回调

****作用****：工具执行时通知 UI 层进行显示。

```
public interface ToolDisplayCallback {
    void onToolStart(String toolName, String summary);
    void onToolEnd(String toolName, boolean success, String resultSummary);
    default void onToolEnd(String toolName, boolean success, String resultSummary, String fullContent);
}
```

四、com.javagent.tools 包

Tool —— 工具接口

****作用****：所有工具的统一规范，策略模式的体现。

****方法****：

- definition：返回工具的元信息定义
- execute：接收参数 Map，执行操作，返回执行结果
- configure(Path)：可选，配置工作区根目录

ToolDefinition —— 工具定义

****作用****：描述工具的元信息，用于告诉 AI 可用工具和审批判断。

****字段****：

- name：工具名称
- description：功能描述
- parameterDescriptions：每个参数的含义
- parameterTypes：每个参数的类型
- requiredParameters：必填参数集合
- requiresApproval：是否需要用户审批
- readOnly：是否只读
- destructive：是否破坏性操作
- aliases：工具别名列表

ToolExecutionResult —— 工具执行结果

****作用****: 工具执行后的返回值，包含结果内容和是否出错标志。

****工厂方法****: success / error

ToolRegistry —— 工具注册中心

****作用****: 管理所有可用工具，支持按名称和别名查找。

****核心数据结构****: 两个 Map，分别按工具名和别名存储

register

1. 将工具按名称（小写）存入名称 Map
2. 遍历别名，逐个存入别名 Map

find

1. 先按名称查找
2. 找不到再按别名查找
3. 都找不到返回空

definitions

- 返回所有工具的定义列表，用于告诉 AI 有哪些工具

listTools

- 生成格式化的工具列表文本，包含名称、别名、描述、审批标志

ToolProvider —— SPI 插件接口

****作用****: 外部工具插件的扩展点，通过 ServiceLoader 自动发现。

```
public interface ToolProvider {
    List<Tool> tools();
}
```

在 META-INF/services/com.javagent.tools.ToolProvider 中注册即可。

FileToolSupport —— 文件工具辅助类

****作用****: 各文件工具共用的参数提取和路径处理方法，避免代码重复。

****方法****:

- stringValue: 从 Map 提取字符串值，null 返回空串
- intValue: 从 Map 提取整数值，兼容 Number 和 String 类型
- booleanValue: 从 Map 提取布尔值，兼容 Boolean 和 String 类型
- normalizePath: 规范化路径，去除冗余的 "." 和 ".."
- isBinary: 检测文件是否为二进制文件（只检测 null 字节）

- checkInsideWorkspace: 检查路径是否在工作区内

ReadFileTool —— 读文件工具

****作用****: 读取文本文件内容, 支持指定起始行和行数限制。

****安全标记****: 不需审批、只读、非破坏性

****安全限制****:

- 文件大小超过 256KB 拒绝读取
- 二进制文件拒绝读取

execute

1. 提取并验证路径参数
2. 规范化路径
3. 检查文件是否存在且是普通文件
4. 检查文件大小和是否为二进制
5. 读取文件全部行
6. 根据 offset 和 limit 截取指定范围
7. 构建带行号的输出格式
8. 如果还有更多行, 显示省略提示

GrepTool —— 文本搜索工具

****作用****: 在文件中递归搜索匹配正则表达式的文本。

****安全标记****: 不需审批、只读、非破坏性

****限制****: 最多扫描 200 个文件、最多返回 100 个匹配

execute

1. 提取并验证搜索模式
2. 提取搜索路径 (默认当前目录)
3. 编译正则表达式 (支持大小写敏感/不敏感)
4. 如果目标是文件 → 直接搜索该文件
5. 如果目标是目录 → 递归遍历, 跳过 .git、target 等目录
6. 对每个文件逐行匹配, 输出匹配行 (文件路径:行号:内容)
7. 统计扫描文件数和匹配数

ListDirectoryTool —— 目录列表工具

****作用****: 列出指定目录下的文件和子目录。

****安全标记****: 不需审批、只读、非破坏性

execute

1. 提取路径参数（默认当前目录）
2. 检查路径是否存在且是目录
3. 根据是否递归选择遍历方式
4. 过滤掉自身、按名称排序、限制条目数
5. 输出 [D] 标记目录、[F] 标记文件
6. 递归模式显示相对路径，非递归模式只显示文件名

WriteFileTool —— 写文件工具

****作用****: 向文件写入文本内容，支持覆盖或追加模式。

****安全标记****: 需要审批、非只读、破坏性操作

****限制****: 内容超过 100,000 字符拒绝写入

execute

1. 提取路径和内容参数
2. 检查内容大小
3. 规范化路径
4. 检查目标不是目录
5. 如果文件已存在且是二进制文件，拒绝覆盖
6. 创建父目录（如不存在）
7. 根据追加/覆盖模式选择写入方式
8. 返回写入结果（字符数、路径、模式）

EditTool —— 精确文本替换工具

****作用****: 通过 old_string 精确匹配文件内容并替换为 new_string。

****安全标记****: 需要审批、非只读、非破坏性

****别名****: replace, sed

execute

1. 提取 path、old_string、new_string 参数
2. 检查文件是否存在且是文本文件
3. 读取文件全部内容
4. 搜索 old_string 的所有匹配位置
5. 如果没有匹配 → 尝试模糊查找，提示类似文本位置

6. 如果有多个匹配且 `replace_all=false` → 报错要求更精确的上下文
7. 执行替换
8. 写回文件
9. 构建 diff 预览（红色删除行、绿色新增行）
10. 返回编辑统计

DeleteFileTool —— 删除文件工具

****作用****：删除指定路径的普通文件。

****安全标记****：需要审批、非只读、破坏性操作

execute

1. 提取路径参数
2. 规范化路径
3. 检查文件是否存在
4. 检查是否是普通文件（不支持删除目录）
5. 执行删除
6. 返回删除结果

BashTool —— Shell 命令工具

****作用****：在系统 Shell 中执行命令，返回输出结果。

****安全标记****：需要审批、非只读、破坏性操作

****安全限制****：

- 默认禁用，需用户手动开启
- 内置 10 类危险命令检测（正则匹配）
- 默认超时 10 秒
- 输出超过 50,000 字符自动截断
- 跨平台：Windows 用 `cmd.exe /c`，Unix 用 `/bin/bash -lc`

execute

1. 提取命令参数
2. 检查命令是否为危险命令（10 类正则匹配）
3. 提取超时参数
4. 创建进程构建器（Linux 用 `bash`，Windows 用 `cmd`）
5. 合并标准错误到标准输出
6. 启动进程
7. 逐行读取输出，超过限制则截断

8. 等待进程结束（带超时），超时则强制终止
9. 组装结果（命令 + 输出 + 退出码）
10. 退出码为 0 返回成功，否则返回失败

NetworkTool —— HTTP 请求工具

****作用****：发送 HTTP 请求并获取响应。

****安全标记****：需要审批、非只读、非破坏性

****别名****：http, fetch, request

execute

1. 提取 url、method、timeout 参数
2. 检查 URL 是否为空
3. 默认方法为 GET，超时为 30 秒
4. 创建 HttpClient
5. 构建 HttpRequest
6. 发送请求，获取 HttpResponse
7. 组装结果：状态码、URL、方法、响应头、响应体
8. 返回结果

五、com.javagent.util 包

Terminal —— ANSI 终端格式化

****作用****：提供终端颜色和样式支持，自动检测终端能力。

****方法****：bold、dim、italic、各种颜色方法、lineNumber、diffRemove、diffAdd、filePath、truncate、prompt、isEnabled

MarkdownRenderer —— Markdown 渲染

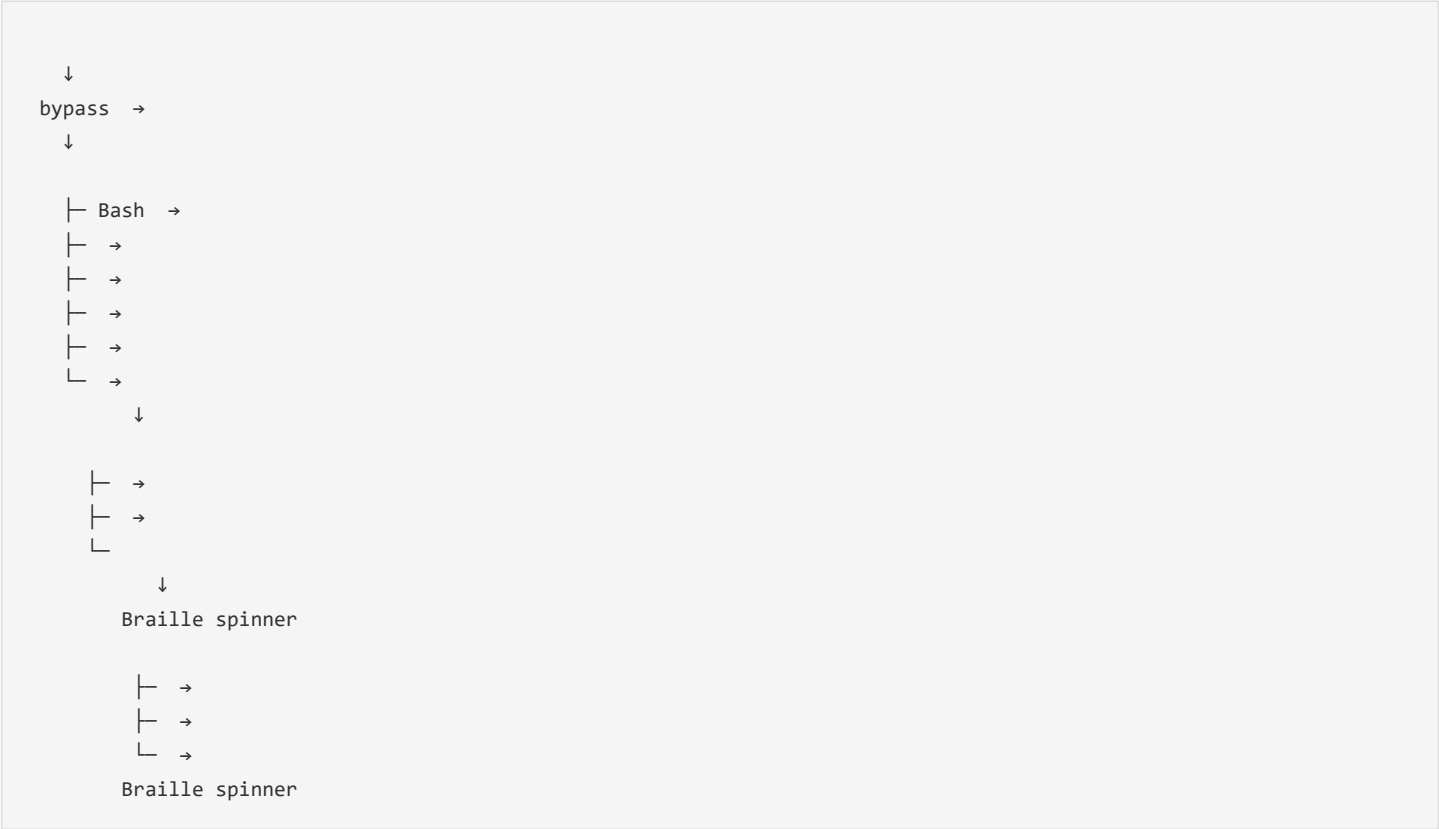
****作用****：将 Markdown 文本转换为 ANSI 彩色输出。

****支持****：标题、代码块、加粗、列表、行内代码

Sanitizer —— 敏感信息过滤

****作用****：在存储到会话历史前脱敏 API 密钥、Token 等敏感信息。

****模式****：sk-xxx、api_key=xxx、token=xxx、Bearer xxx、tp-xxx



安全机制总览

- 1. -
- 2. - bash
- 3. Bypass -
- 4. - 10 rm -rffork bombreverse shell
- 5. - API Key/Token
- 6. - 3
- 7. - API
- 8. -
- 9. API - 3
- 10. - HTTP 400 tools